

DATENSCHUTZBESTIMMUNGEN

=====

Dokumentstand: Aktiver Proof-of-Concept und Entwicklungsstand.

DevBox besitzt einen funktionsfähigen lokalen Launcher, eine grafische Hauptoberfläche, modulare Strukturansichten, Stammdaten- und Dokumentationspflege, Dokumentations-Snapshots, lokale Werkzeugerkennung, eine Repository-Seite sowie erste Bausteine für den DevBox-spezifischen Veröffentlichungsprozess.

Der deskNode-Bereich ist aktiv in DevBox integriert. Supervisor und Daemon können aus der GUI gestartet und gestoppt werden; deren Konsolenausgabe wird im deskNode-Log angezeigt. Die Produktversion kann aus den Stammdaten bearbeitet werden. UX-Themes können benannt, angelegt, gelöscht, dupliziert, umbenannt und gespeichert werden. Nach dem Speichern werden die deskNode-Produktdaten aktualisiert.

DeskNode verfügt im aktuellen Proof of Concept über eine gekoppelte Worker-Architektur. Tapo, FRITZ!DECT und Shelly werden lokal über eigene venmods entdeckt und überwacht; bei unterstützten Geräten werden Schaltzustände und Leistungswerte verarbeitet. Der Daemon synchronisiert die venmod-Daten in einen globalen Gerätebestand, hält einen initialen Sicherheitslesezyklus ein und startet die Hauptoberfläche erst nach der globalen Erst-Synchronisierung. Die Strukturverwaltung unterstützt mehrere Gebäude-Wurzeln, pro Gebäude einen vollständigen Geräte-Pool und zusätzliche additive Zuordnungen zu Räumen oder anderen Strukturgliedern.

Die Symbolverwaltung ist als funktionsfähige Katalogoberfläche vorhanden. Sie verwaltet deskNode-Gerätekategorien und Verbrauchergeräte über Auswahlmenüs sowie Dialoge zum Anlegen, Bearbeiten und Löschen. Neue Geräte erhalten eine einzelne PNG-Quelle, die unter ihrer record_id als symbol_source_<record_id>.png abgelegt wird. Nach jeder erfolgreichen Katalogänderung wird create_manufacturer_db.py ausgeführt, damit mnfctr_db.r0b die aktuellen Theme-, Kategorien- und Gerätedaten enthält.

Der Graphic-Pack-Build ist als Proof of Concept funktionsfähig. Er erzeugt aus Symbolquellen und UX-Themes vorbereitete Grafikzustände für Verbraucher. Die aktuelle deskNode-Laufzeit nutzt bereits Sprach-, Theme- und Grafikdaten; Asset-Auswahl, weitere Symbolabdeckung und visuelle Zustände werden weiterentwickelt.

DevBox und deskNode sind keine fertigen Endnutzer- oder Release-Versionen. Besonders Credential-Speicherung, Langzeitstabilität größerer Gerätebestände, herstellerspezifische Sonderfälle, Tests, Packaging und Plattformportierung sind noch offene Entwicklungsaufgaben.

Erstveröffentlichung: Verantwortliche Stelle / Herausgeber: Markus Walloner Name: Markus Walloner Land: Germany (DE)

1. Gegenstand

Diese Datenschutzbestimmungen beziehen sich auf die in der begleitenden Projektdokumentation beschriebene Software bzw. das beschriebene Projekt.

Kurzbeschreibung: DevBox ist die lokale Entwicklungszentrale der CYXnTrol Development Platform. Die PySide6-Oberfläche bündelt Projektstruktur, Stammdaten, Dokumentation, lokale Werkzeuge, Produktmodule, Repository-Vorbereitung und wiederkehrende Entwicklungsabläufe.

Mit deskNode ist ein aktives Produktmodul integriert: ein lokaler Geräte- und Strukturhub für Smart-Plugs und angeschlossene Verbraucher. deskNode wird über austauschbare venmods für unterstützte Hersteller und Protokolle erweitert.

DevBox ist kein Endnutzerprodukt. Es ist eine interne Arbeitsumgebung, in der Anforderungen, Datenmodelle, UX-Entscheidungen, Schnittstellen, Builds und Veröffentlichungsstände nachvollziehbar vorbereitet und geprüft werden.

2. Datenschutzbestimmungen

DATENSCHUTZHINWEIS - ENTWURFSFASSUNG

1. Zweck und Geltungsbereich

Dieser Datenschutzhinweis beschreibt den derzeit vorgesehenen lokalen Betrieb von DevBox. DevBox ist eine lokale Entwicklungsumgebung. Nach dem aktuellen Konzept nutzt die Anwendung keine Telemetrie, Werbung, Analyse-Dienste oder proprietären Cloud-Dienste ohne ausdrückliche Nutzeraktion.

2. Verantwortliche Stelle und Kontakt

Verantwortliche Stelle für den DevBox-Entwicklungsstand ist Markus Walloner innerhalb von CYX Labs. Fragen zu diesem Projekt können über den im jeweiligen Repository oder in begleitenden Projektdokumenten angegebenen Kontaktweg gerichtet werden.

3. Lokal verarbeitete Daten

DevBox verarbeitet insbesondere lokal:

- Pfade zum Projektroot, zu Projektdateien und lokalen Werkzeugen;
- Inhalte lokaler SQLite-Datenbanken mit Produkt-, Hersteller-, Dokumentations-, Repository-, Struktur-, UX-Theme-, Sprach-, Kategorien- und Verbrauchhermetadaten;
- lokale PNG-Quellen für Verbraucher, abgeleitete deskNode-Produktdatenbanken und vorbereitete Grafikpakete;
- Runtime- und Fehlerprotokolle in logfile.r0b sowie produktbezogene Konsolenausgaben;
- erkannte Pfade zu Inkscape und GIMP in locdata.r0b;
- Versionsnummern, Theme-Namen, RGBA-Werte, Kategorienamen, Geräteschlüssel und weitere Oberflächeneinstellungen;
- deskNode-Geräteinventar, lokale Netzwerkadressen, Gerätezustände, Leistungswerte, Strukturzuordnungen und zeitbezogene Verbrauchsprotokolle;
- Zugangsdaten für Herstellerpfade, soweit diese für eine vom Nutzer ausgelöste lokale Geräteanmeldung erforderlich sind;

- Commit-Texte sowie Bildpfade und Bilddateien für ausdrücklich ausgelöste Repository-Vorgänge;
- technische Ausgaben lokaler Teilprozesse, beispielsweise Python, Git, Inkscape, GIMP, Daemon, Worker oder produktbezogene Skripte.

Diese Daten werden grundsätzlich lokal in Projektordnern, temporären Arbeitsbereichen oder unter AppData verarbeitet und dienen der jeweils vom Nutzer gewählten DevBox- oder deskNode-Funktion.

4. DeskNode, lokale Netzkommunikation und Credentials

DeskNode kommuniziert im aktuellen Proof of Concept mit unterstützten Geräten oder Gateways im lokalen Netzwerk, etwa über Tapo-, FRITZ!DECT- oder Shelly-Pfade. Dabei können lokale IP-Adressen, Gerätekennungen, Status- und Leistungswerte verarbeitet werden. Zugangsdaten werden nicht in Konsolenausgaben oder vorgesehenen Worker-Ereignissen ausgegeben. Der aktuelle Entwicklungsstand besitzt jedoch noch keinen plattformübergreifend verschlüsselten Credential-Vault. Lokal persistierte Zugangsdaten dürfen daher nicht als gehärteter Secret Store betrachtet werden; vor öffentlichem oder produktivem Einsatz ist eine sichere Vault-Lösung erforderlich.

5. Temporäre Arbeitsbereiche

Für Dokumentationsexporte, Grafik-Builds, Repository-Vorbereitung, deskNode-Runtime und ähnliche Vorgänge erstellt DevBox eindeutige temporäre Arbeitsordner. Dort können Kopien von Projektdateien, Zwischenstände, Grafiken, Masken, erzeugte Dokumente, Laufzeitdaten und vorbereitete Assets verarbeitet werden. Die Anwendung versucht, diese Arbeitsbereiche nach Abschluss zu entfernen. Bei fehlgeschlagenem Cleanup können einzelne temporäre Dateien bis zu einer späteren manuellen oder systemseitigen Bereinigung bestehen bleiben.

6. Externe Programme

DevBox kann nach ausdrücklicher Nutzeraktion lokale Programme wie Inkscape, GIMP, Git, Git Credential Manager oder Installer starten. Nach dem Start können deren eigene Datenverarbeitung und Nutzungsbedingungen gelten. DevBox speichert keine Git-Passwörter oder Zugriffstokens als eigene Repository-Zugangsdaten.

7. Repository-Vorgänge und Datenübertragung

Wird ein Push-to-Git-Vorgang ausdrücklich ausgelöst, überträgt Git die vom Nutzer ausgewählten Repository-Inhalte, Commit-Informationen und technisch erforderliche Verbindungsdaten an den jeweiligen Repository-Anbieter. Empfänger, Sichtbarkeit und Branch ergeben sich aus der vom Nutzer gepflegten Repository-Konfiguration. Der Nutzer ist dafür verantwortlich, keine personenbezogenen oder vertraulichen Daten unbeabsichtigt zu veröffentlichen.

8. Rechtsgrundlage und Prüfung

Dieser Text ist ein technischer Entwurf und keine individuelle Rechtsberatung. Vor öffentlicher oder kommerzieller Verbreitung muss er an tatsächliche Datenflüsse, Kontaktangaben, mögliche Rechtsgrundlagen, Auftragsverarbeitungen, internationale

Übermittlungen und das anwendbare Datenschutzrecht angepasst und rechtlich geprüft werden.

3. Projektkontext

Zweck: DevBox soll wiederkehrende und fehleranfällige Entwicklungsarbeit in nachvollziehbare lokale Werkzeuge überführen. Dazu gehören insbesondere die Pflege von Produkt- und Herstellerdaten, strukturierte Dokumentation, vorbereitete Veröffentlichungsschritte, die Integration lokaler Kreativprogramme sowie die kontrollierte Ausführung produktbezogener Entwicklungsfunktionen.

Die Anwendung schafft eine gemeinsame Arbeitsgrundlage für Produkte der CYXnTrol Development Platform. Statt Abläufe nur als Erinnerung, Ordnerkonvention oder Sammlung einzelner Konsolenbefehle zu halten, werden sie als Daten, Skripte, GUI-Funktionen und überprüfbare Prozessketten festgehalten.

Für deskNode dient DevBox als Entwicklungs- und Konfigurationsoberfläche für Produktversionen, UX-Themes, Gerätekategorien, Verbrauchersymbole, Sprachressourcen und vorbereitete Grafikpakete. Die deskNode-Laufzeit soll daraus reproduzierbare Daten und vorbereitete Assets erhalten. Die venmod-Architektur soll zudem neue lokale Gerätepfade ergänzen können, ohne den Daemon oder bereits funktionierende Herstellerpfade unnötig umzubauen.

Konfiguration: Die Konfiguration ist in mehrere Ebenen getrennt.

Projektroot: Der Projektroot wird über die .root-Datei gefunden. Er ist die maßgebliche Quelle für Skripte, Ressourcen, Formarchive, Installer, globale Fonts und die zentrale DevBox-Datenbank.

Zentrale DevBox-Stammdaten: resources/organization/devbox_db.r0b enthält Hersteller-, Produkt-, Dokumentations-, Struktur-, Repository-, UX-Theme- und Symbolkataloginformationen. Die Tabelle "ux-deskNode" enthält benannte Theme-Datensätze mit record_id und theme_name. Die Tabellen desknode_consumer_device_categories und desknode_consumer_devices enthalten die dauerhaften technischen Kategorien und Verbrauchergeräte. Schemaerweiterungen sollen vorhandene Datensätze erhalten und fehlende Felder über Migration ergänzen.

DeskNode-Produktdaten: applications/deskNode/data/mnfctr_db.r0b enthält master_data, ux_themes, consumer_device_categories und consumer_devices. applications/deskNode/data/lan.r0b enthält das Sprachpaket language_package. fonts.r0b und graphic_items.r0b dienen als Produktarchive für die Laufzeit. create_manufacturer_db.py aktualisiert die abgeleitete Manufakturdatenbank nach erfolgreichen Änderungen an Version, Theme, Kategorie oder Verbrauchergerät.

DeskNode-AppData und Laufzeit: Aus master_data wird der lokale AppData-Pfad gebildet, aktuell typischerweise %APPDATA%/CYX Labs/CYXnTrol/deskNode. settings.r0b speichert die gewählte UI-Sprache und das aktive UX-Theme sowie vom Supervisor benötigte Laufzeitinformationen. devices.r0b enthält den globalen Gerätebestand und additive Strukturzuordnungen. log_device_power.r0b protokolliert aufgelöste Leistungswerte. Jeder aktive Daemon-Lauf besitzt zusätzlich einen eindeutigen Temp-Ordner mit

runtime_state.r0b und getrennten flüchtigen venmod-Datenbanken.

Gerätestruktur: Eine frische Installation erhält mindestens eine Gebäude-Wurzel. Jedes Gebäude besitzt intern einen festen Geräte-Pool als vollständige Gebäudeübersicht. Geräte können im Pool verbleiben und zusätzlich räumlichen oder funktionalen Gliedern zugeordnet werden. Die Strukturverwaltung unterstützt mehrere unabhängige Gebäude-Wurzeln.

Venmods: Ein venmod wird über eine coupler.py gefunden und muss einen gültigen Kopplungsvertrag melden. Dieser beschreibt Name, Workerstart, flüchtige Datenbank, Gerätetabelle sowie Upstream- und Downstream-Spaltenzuordnungen. Der Daemon validiert diese Angaben vor dem Workerstart. Tapo, FRITZ!DECT und Shelly sind die derzeit eingebundenen lokalen Pfade.

Credentials und Sicherheitsstand: Zugangsdaten werden nur für Herstellerpfade benötigt, die eine Anmeldung verlangen. Sie werden nicht in Logs ausgegeben. Der aktuelle Proof of Concept besitzt jedoch noch keinen plattformübergreifenden verschlüsselten Credential-Vault; lokal persistierte Entwicklungsdaten dürfen daher nicht als gehärteter Secret Store behandelt werden. Die geplante Vault-Lösung soll als gemeinsame Plattformkomponente entstehen, nicht als venmod-spezifische Sonderlösung.

Oberflächengestaltung: Farben werden als achtstellige RGBA-Hexwerte ohne # gespeichert, zum Beispiel 00e4ffff. Schriftdateien werden rekursiv aus resources/fonts gelesen und als projektroot-relative Pfade gespeichert. Schriftrollen umfassen große Überschriften, Bereichsüberschriften, Standardtext, Schaltflächentext, Eingabefeldtext, Statusmeldungen und Protokolltext. Weitere Theme-Werte betreffen Schriftgrößen, Schriftschnitt, Unterstreichung, Konturen, Radien und Zustandsfarben.

Werkzeuigerkennung und Repository-Daten: Gespeicherte Pfade werden beim Start geprüft. Ungültige Inkscape- oder GIMP-Pfade werden erneut gesucht. Repository-URL und Branch werden pro Produkt in product_credentials gepflegt. Git-Zugangsdaten werden nicht in DevBox gespeichert. Für den DevBox-Push wird ein temporärer Veröffentlichungsroot erzeugt; der aktuelle Exportumfang wird durch den kontrollierten Push-Prozess festgelegt.

Technologie: DevBox verwendet derzeit vor allem folgende Technologien:

- Python als Kernsprache für Launcher, Prozesslogik, Datenaufbereitung, Automationen und produktbezogene Werkzeuge.
- PySide6 für die lokalen grafischen Benutzeroberflächen von DevBox und deskNode.
- SQLite für zentrale Projektstammdaten, deskNode-Produktdaten sowie lokale Runtime-, Geräte-, Einstellungs-, Sprach- und Protokolldaten.
- python-kasa für die lokale Tapo-Erkennung und Gerätekommunikation im Tapo-venmod.
- Lokale HTTP-, XML- und RPC-Kommunikation für FRITZ!DECT/AHA- und Shelly-Pfade, soweit der jeweilige venmod sie implementiert.
- Einen lokalen Worker-Control-Hub und getrennte Workerprozesse für die venmod-Anbindung.
- openpyxl als Übergangs- oder Importwerkzeug für Tabellen, nicht als zentrale Stammdatenquelle.

- ReportLab und svglib für die lokale Erzeugung gestalteter PDF-Dokumente.
- Git und Git Credential Manager für Repository-Vorgänge und Authentifizierung.
- Inkscape für Skalierung, SVG-Arbeit und Vektorisierung von Masken.
- GIMP 3 für nichtinteraktive Bildbearbeitung und vorbereitende Maskenschritte.
- XML-/SVG-Verarbeitung mit Python-Standardbibliotheken für die Bereinigung generierter Masken.
- C#-Generierung, .NET SDK, WiX und perspektivisch weitere Packaging-Werkzeuge für Build- und Installer-Prozesse.
- temporäre Arbeitsbereiche unter dem System-Temp-Verzeichnis für kontrollierte Kopien, Grafik-Builds, Exporte, Daemon-Laufzeiten und Veröffentlichungsvorbereitung.

Die technische Struktur trennt GUI, Funktionsskripte, Subscripts, Datenquellen, Produktdatenbanken, externe Werkzeuge, globale Gerätehaltung und venmod-spezifische Runtime möglichst klar. Der deskNode-Symbolkatalog verknüpft SQLite-Datensätze mit PNG-Quellen über stabile record_id-Werte, während der Graphic-Pack-Build die daraus resultierenden visuellen Varianten erzeugt.

Repository-Hinweis: Das DevBox-Repository repräsentiert die CYXnTrol Development Plattform selbst, nicht ein fertiges Endnutzerprodukt. Es dient als Entwicklungsstand, transparente Arbeitsprobe und Grundlage für die Pflege der Plattform.

Der veröffentlichte Stand wird nicht direkt aus dem produktiven Projektroot kopiert. Für DevBox wird ein gesonderter bereinigter root_dir-Stand erzeugt. Er enthält vorgesehenen Quellcode, Dokumente und Assets, während lokale Laufzeitdaten, temporäre Reste, Installer, Datenbank-WAL-Dateien und nicht veröffentlichte Daten außerhalb des Veröffentlichungspakets bleiben.

Der derzeitige DevBox-Push behandelt applications als bewusstes Auslieferungsfenster: Der Bereich wird zunächst bereinigt. Danach wird applications/deskNode/resources/scripts einschließlich enthaltener Dateien in den Veröffentlichungsstand kopiert.

__pycache__-Ordner sowie .pyc- und .pyo-Dateien bleiben ausgeschlossen. Dieser aktuelle Umfang ist ein DevBox-spezifischer Veröffentlichungsprozess und noch kein vollständiger deskNode-Produktrelease. Bevor deskNode einschließlich GUI, Logik und venmods als eigenes Repository oder Release veröffentlicht wird, muss dessen Exportumfang ausdrücklich definiert und getestet werden.

DevBox wird in einem KI-gestützten, iterativen Workflow entwickelt. Anforderungen, Architektur, Datenmodelle, UX-Entscheidungen, Testfälle und Abnahmen werden durch den Projektverantwortlichen gesteuert. Die Implementierung entsteht mit KI-gestützten Entwicklungswerkzeugen und wird gegen die definierten Funktionsanforderungen geprüft.

Copyright (c) 2026 Markus Walloner